



Smart Livestock Management System

Project Title	CattleCloud — Smart Livestock Management System
Project Type	Full-Stack Web Application
Session	2025–2026
Report Date	May 2026
Repository	minorproject (Monorepo)

Developed by the Project Team

Under the Guidance of

CERTIFICATE

This is to certify that the project report entitled "**CattleCloud — Smart Livestock Management System**" submitted by _____ to _____ **University/College** in partial fulfillment of the requirements for the award of _____ is a record of bonafide work carried out under my supervision.

The matter embodied in this report has not been submitted earlier for the award of any other degree/diploma to the best of my knowledge.

Guide Name

Designation _____

Signature: _____

Date: _____

Head of Department

Department _____

Signature: _____

Date: _____

DECLARATION

We hereby declare that the project work entitled "**CattleCloud — Smart Livestock Management System**" submitted to _____ **Institution** is an authentic record of our own work carried out during the academic year **2025–2026** under the supervision of _____.

This work has not been submitted elsewhere for a degree or diploma. We have given due acknowledgments wherever materials from other sources have been used.

Student Name			

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide _____ for continuous support, valuable suggestions, and encouragement throughout the development of CattleCloud.

We thank the **Head of the Department** and all faculty members for providing laboratory facilities and academic guidance.

We are grateful to our families and peers for their motivation during the implementation and testing phases of this application.

Finally, we acknowledge the open-source communities behind **React**, **Node.js**, **Express**, **MongoDB**, and **Vite**, whose tools made this project feasible within an academic timeline.

Team Members

ABSTRACT

Livestock farming remains a critical sector in rural economies, yet many small and medium farmers continue to rely on informal record-keeping. Missed vaccination schedules, inconsistent milk yield tracking, and poor visibility into farm expenses directly affect productivity and income. **CattleCloud** addresses these challenges through a browser-based **Smart Livestock Management System** that centralizes operational data in one secure dashboard.

The system is implemented as a **three-tier web application**. The **presentation tier** is a React 19 single-page application built with Vite, offering bilingual support in **English and Hindi**. The **application tier** is a RESTful API developed with Express 5 and Node.js, using JSON Web Tokens (JWT) and bcrypt for authentication. The **data tier** is MongoDB, accessed through Mongoose schemas for users, animals, milk records, vaccinations, and financial transactions.

Key capabilities include farmer registration and login, per-user data isolation, animal registry with Indian cattle and buffalo breeds, milk production logging with revenue estimation at Rs. 70 per litre, vaccination scheduling with automatic booster date calculation, and an expense/profit ledger with net farm profit/loss summary. A marketing landing page, Chart.js-based farmer dashboard, and Netlify deployment configuration complete the solution.

The project demonstrates practical full-stack engineering suitable for academic evaluation. Certain features remain in prototype state — department-level analytics dashboard, server-integrated password recovery, and environment-based API configuration — which are documented as future enhancements.

The repository is organized as a monorepo: `backend/` hosts the Express API and Mongoose models; `smart-livestock-dashboard/` contains the React client built with Vite; `data/` holds legacy JSON samples from early prototyping. Farmers interact through standard HTML forms and tables; administrators (future scope) would use aggregated dashboards once backend analytics endpoints are implemented.

From a software engineering perspective, the project applies separation of concerns, stateless authentication, and multi-tenant data isolation at the query level. These patterns mirror production systems while keeping the codebase small enough for code review during viva and minor project evaluation.

Keywords: *Livestock management, MERN-style stack, React, Express, MongoDB, JWT, dairy farming, vaccination tracking, bilingual web application.*

TABLE OF CONTENTS

1	Introduction
2	Literature Survey & Existing Systems
3	Problem Definition & Objectives
4	System Analysis
5	System Design
6	Technology Stack
7	Implementation — Backend
8	Implementation — Frontend
9	Database Design
10	Security & Authentication
11	User Interface & Screen Documentation
12	Algorithms & Business Logic
13	API Reference (Complete)
14	Testing & Validation
15	Deployment & DevOps
16	Results & Discussion
17	Limitations & Known Issues
18	Future Scope
19	Conclusion
A	Appendix A — Project Directory Structure
B	Appendix B — Glossary
C	Appendix C — References
D	Appendix D — Viva Preparation (Question Bank)
E	Appendix E — Translation Module Overview
F	Appendix F — Team Contribution Template

Chapter 1 — Introduction

1.1 Background

India is among the largest milk-producing countries globally, with millions of households engaged in dairy and mixed livestock farming. Despite digital initiatives in agriculture, a significant portion of cattle owners still maintain records manually — notebooks, wall calendars, or verbal memory — for animal identity, milk quantities, veterinary visits, and cash flows.

Manual systems introduce errors and delays:

- A missed **Foot and Mouth Disease (FMD)** booster can affect herd health and marketability.
- Without daily milk logs, farmers cannot identify high-yield versus low-yield animals.
- Expenses scattered across feed, labor, and veterinary bills make **net profitability** unclear.

CattleCloud was conceived as an academic full-stack project to model a digital livestock management solution using modern, industry-relevant technologies.

1.2 About the Project

CattleCloud (repository folder: *smart-livestock-dashboard*) is a web-based livestock management platform targeting individual farmers. After authentication, a farmer can:

- Register and maintain animals with breed, age, gender, purchase cost, and health status.
- Record milk production per animal per day.
- Maintain vaccination history with computed next-due dates.
- Track expenses and profits under categorized ledgers.
- View a dashboard with KPI cards and weekly milk production charts.
- Access a public marketing landing page with feature descriptions and FAQ.

1.3 Scope of the Project

In Scope

- User registration and login against MongoDB.
- JWT-based session tokens for API authorization.
- CRUD operations for animals, milk, vaccinations, and expenses.
- User-scoped data access on the server.
- Bilingual UI strings (English / Hindi).
- Farmer dashboard with Chart.js visualization.
- Profile management with image upload (base64 stored in database).
- Frontend build and Netlify SPA configuration.

Out of Scope (Current Version)

- SMS/email notifications for vaccine reminders.
- Real-time IoT sensor integration (automated milk meters, RFID tags).
- Native Android/iOS applications.
- Multi-farm enterprise hierarchy.
- Fully functional government department dashboard (placeholder only).

1.5 Target Users

User Type		
Farmer	Primary operator managing own herd	Full module access after login
Guest	Visitor exploring product	Landing page only
Department Official	Aggregated regional view	Route exists; static placeholder

1.4 Organization of the Report

This document is structured to follow standard software engineering practice for academic minor projects. Chapters 1–3 establish context, survey, and objectives. Chapters 4–6 cover analysis, design, and technology choices. Chapters 7–9 describe implementation and database design. Chapters 10–13 address security, user interface, algorithms, and API specification. Chapters 14–16 present testing, deployment, and results. Chapters 17–19 discuss limitations, future work, and conclusion. Appendices provide directory structure, glossary, references, and viva preparation material.

1.6 Motivation for Digital Livestock Management

- India produces over 200 million tonnes of milk annually, with a large share from smallholder farms.
- Government schemes promote animal tagging and veterinary outreach, but daily operational records remain farmer-managed.
- A simple web dashboard reduces dependency on memory during peak milking and feeding hours.
- Digital records can be shown to veterinarians or dairy cooperatives without re-typing information.
- Students gain hands-on experience with authentication, REST APIs, and document databases used in industry.

1.7 Project Repository Structure

The monorepo contains three logical parts: **backend/** (Node.js API), **smart-livestock-dashboard/** (React client), and **data/** (JSON seed files from early prototyping, not used at runtime). Environment secrets live in `backend/.env` and must never be committed to public repositories.

Folder	
backend/	Express server, Mongoose models, JWT middleware
smart-livestock-dashboard/	Vite + React SPA, pages, components

Folder	
data/	Sample Animals.json, Milk.json — reference only

Chapter 2 — Literature Survey & Existing Systems

2.1 Traditional Manual Systems

Traditional farm management relies on paper registers. Advantages include zero technology cost and no learning curve. Disadvantages include: no automatic search or filtering, no aggregation without manual calculation, high risk of data loss, and difficult backup and sharing with veterinarians or cooperatives.

2.2 Spreadsheet-Based Systems

Farmers and cooperatives often use Microsoft Excel or Google Sheets. These improve calculability but lack structured validation, role-based access control, mobile-optimized workflows, and integrated per-farmer authentication.

2.3 Commercial Livestock Software

Commercial solutions offer comprehensive modules but are expensive for smallholders, over-engineered for a minor project scope, and dependent on vendor lock-in and training. **CattleCloud** intentionally scopes down to core modules teachable within one academic semester while using the same architectural patterns.

2.4 Comparative Summary

Feature			
Search/filter animals	Poor	Medium	Yes
Per-user cloud storage	No	Manual	Yes (MongoDB)
Vaccine reminder logic	No	Manual	Partial (UI alerts)
Mobile browser access	N/A	Medium	Yes
Auth & privacy	N/A	Weak	JWT + user filter
Cost	Low	Low/Medium	Hosting dependent

2.5 Government and Cooperative Portals

Regional agriculture departments and milk cooperatives operate portals for subsidy registration, bulk milk collection, and scheme enrollment. These systems focus on institutional data rather than a private farm notebook. CattleCloud complements such platforms by giving each farmer a personal ledger for herd health and daily operations without replacing government workflows.

2.6 Technology Trends in Agri-Tech

Trend	
Cloud databases (MongoDB Atlas)	Scalable persistence via MONGO_URI
JWT for SPAs	Matches React + Express architecture
Bilingual rural UI	English/Hindi via translations.js
Data visualization	Chart.js weekly milk bar chart
PWA / offline	Future enhancement for low connectivity
IoT milk sensors	Out of scope; manual entry in current version

2.7 Gap Analysis

Existing manual and spreadsheet methods fail to combine **authentication**, **per-user privacy**, **vaccination scheduling logic**, and **unified milk plus finance reporting** in one browser application. CattleCloud fills this gap for demonstration and farmer pilot use, while acknowledging that enterprise herd software offers deeper breeding genetics and inventory modules.

Chapter 3 — Problem Definition & Objectives

3.1 Problem Statement

Small and medium livestock farmers lack an affordable, simple, and language-accessible digital system to record animals, milk yield, vaccinations, and expenses in one place, leading to operational inefficiency and financial opacity.

3.2 Need for the System

- **Health compliance** — Timely vaccinations reduce disease outbreaks.
- **Productivity** — Milk logs help identify best-performing animals.
- **Financial clarity** — Combined expense and profit entries support net margin understanding.
- **Digital literacy** — Simple forms and Hindi language lower adoption barriers.
- **Academic learning** — Demonstrates end-to-end full-stack development.

3.3 Primary Objectives

ID		
O1	Secure user accounts	Register/login works; passwords hashed
O2	Animal registry	Add/list/delete animals per user
O3	Milk tracking	Records stored and shown in table + dashboard
O4	Vaccination records	Schedule stored with next due date
O5	Financial ledger	Expense/profit entries with net summary
O6	Dashboard analytics	KPI cards + weekly bar chart
O7	Bilingual UI	Toggle EN/HI on key screens

3.4 Constraints

- Development time bounded by academic calendar.
- Single MongoDB instance (no sharding).
- Hardcoded milk price (Rs. 70/L) for revenue estimation.
- API base URL fixed to localhost:5000 in development.

3.5 Secondary Objectives

ID		
S1	Department aggregate dashboard	Placeholder only
S2	Email-based password reset	Not implemented
S3	Production full-stack deploy	Frontend Netlify config only
S4	Automated unit tests	Not implemented

3.6 Working Hypothesis

If farmers maintain digital records for milk and vaccinations, they will spend less time reconciling paper notes and can identify underperforming animals faster—assuming basic smartphone or shared-computer access and stable internet during data entry sessions.

Chapter 4 — System Analysis

4.1 Functional Requirements

ID	
FR-1.1	User shall register with name, email, password.
FR-1.2	User shall login and receive JWT.
FR-1.3	Protected pages shall require authentication flag.
FR-2.1	User shall add animal with ID, breed, age, gender, cost, health.
FR-2.2	User shall view list of own animals only.
FR-2.3	User shall delete own animals.
FR-2.4	User shall search animals by ID substring.
FR-3.1	User shall log milk quantity per animal per date.
FR-3.2	System shall display per-record income at configured rate.
FR-4.1	User shall select vaccine from predefined list.
FR-4.2	System shall compute upcoming booster date from rules.
FR-4.3	UI shall warn if booster due within 3 days.
FR-5.1	User shall record type Expense or Profit.
FR-5.2	Categories shall depend on type selected.
FR-5.3	System shall compute totals and net profit/loss.
FR-6.1	Display animal count, milk metrics, farm score.
FR-6.2	Display weekly milk bar chart.

4.2 Non-Functional Requirements

ID		
NFR-1	Performance	API response < 2s on LAN for typical datasets
NFR-2	Usability	Forms with labels; bilingual labels
NFR-3	Security	Passwords hashed; JWT on protected routes
NFR-4	Scalability	User-scoped queries (horizontal scaling possible later)
NFR-5	Maintainability	Modular route files per domain
NFR-6	Portability	Browser-based; OS independent

4.3 Feasibility Study

Area	
Technical	High — React and Node ecosystems are mature; MongoDB free tier available.
Economic	High for demo — Open-source stack; Netlify/MongoDB Atlas free tiers.
Operational	Medium — Requires farmer internet access; minimal training needed.

4.4 Stakeholder Analysis

Stakeholder		
Farmer	Daily operations	Primary user of all modules
Veterinarian	Health history	Indirect — farmer shows records
Dairy cooperative	Milk volume	Future API integration
Project evaluator	Code quality, demo	Report and viva
Developer team	Learning MERN patterns	Build and maintain code

4.5 As-Is vs To-Be Workflow

As-Is (Manual)

Farmer writes in notebook → manual sums at month end → optional Excel copy → difficult to search historical vaccination dates.

To-Be (CattleCloud)

Farmer opens browser → logs in → enters data in forms → MongoDB stores records → dashboard aggregates KPIs and charts instantly → delete/search available per module.

4.6 Module-wise Requirement Traceability

Module		
Auth	FR-1.x	/api/auth
Animals	FR-2.x	/api/animals
Milk	FR-3.x	/api/milk
Vaccination	FR-4.x	/api/vaccinations
Expenses	FR-5.x	/api/expenses
Dashboard	FR-6.x	GET animals + milk
Profile	—	/api/users

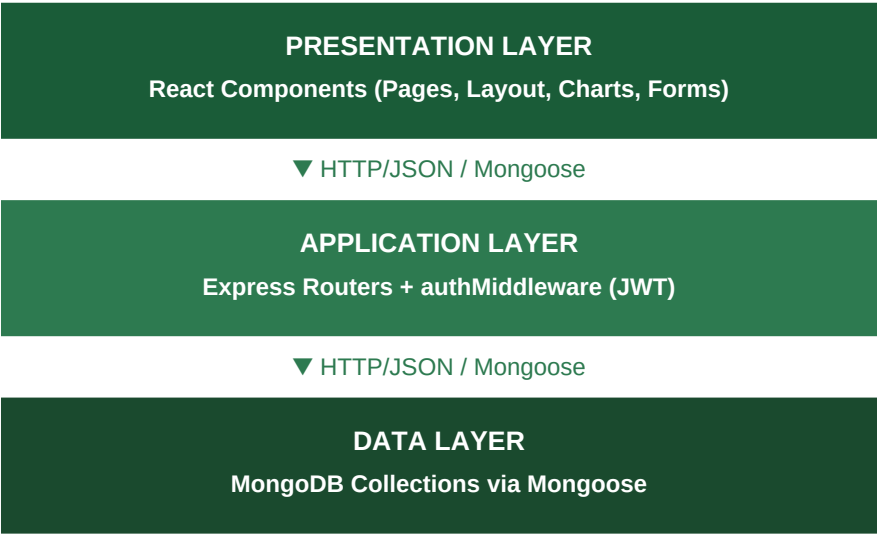
4.7 Assumptions

- Farmers have basic literacy and can use web forms.
- Internet is available at least once per day for sync.
- Animal IDs are unique per farmer (not globally).
- Milk price Rs. 70/L is acceptable for demo revenue estimates.
- Dates are entered in local calendar and stored as strings.

Chapter 5 — System Design

5.1 Architectural Pattern

The project uses **Client-Server architecture** with **REST API** communication. It aligns with a simplified **MERN** stack (MongoDB, Express, React, Node) with the frontend powered by Vite rather than Create React App.



5.2 Module Decomposition

Module	
Auth	Identity lifecycle
Users	Profile read/update image
Animals	Herd registry
Milk	Production diary
Vaccinations	Preventive health schedule
Expenses	Farm accounting
i18n	Translation dictionary
Layout	Shell navigation

5.3 Design Decisions

Decision		
MongoDB documents	Flexible fields per record type	No strict SQL joins
JWT vs sessions	Stateless API, SPA-friendly	Token storage security
fetch vs axios	Simplicity in pages	Duplicated base URL
Delete-only (no PUT)	Reduced scope	Cannot edit records in-place
Base64 profile images	No separate file server	Large documents in DB

5.4 Login Sequence (Narrative)

- **Step 1:** User submits email and password on Login page.

- **Step 2:** React sends POST /api/auth/login with JSON body.
- **Step 3:** Server finds User by email; bcrypt.compare validates password.
- **Step 4:** Server signs JWT with { id: user._id }, expiry 1 day.
- **Step 5:** Client stores token, user name, and auth flag in localStorage.
- **Step 6:** React Router navigates to /farmer-dashboard.
- **Step 7:** Dashboard fetch calls include Authorization: Bearer token header.

5.5 Component Hierarchy (Frontend)

App.jsx

■■■ Public routes: Landing, Login, Register, ForgotPassword

■■■ PrivateRoute

■■■ Layout (Topbar, Sidebar, Footer)

■■■ FarmerDashboard

■■■ Animal, Milk, Vaccination, Expenses

■■■ DepartmentDashboard (static)

Chapter 6 — Technology Stack

6.1 Frontend Technologies

Technology		
React	19.2	Component-based UI
Vite	7.3	Dev server, HMR, production bundling
React Router	7.13	Client-side routing
Chart.js	4.5	Chart rendering engine
react-chartjs-2	5.3	React wrapper for Chart.js
Recharts	3.7	Optional/alternate charts
Lucide React	0.575	Icons
ESLint	9.x	Static analysis

6.2 Backend Technologies

Technology		
Node.js	LTS	JavaScript runtime
Express	5.2	HTTP routing, middleware
Mongoose	9.2	ODM for MongoDB
jsonwebtoken	9.0	Token issue/verify
bcryptjs	3.0	Password hashing
cors	2.8	Cross-origin headers
dotenv	17.3	Environment configuration

6.3 Deployment Stack

Component		
Frontend static hosting	Netlify	netlify.toml, public/_redirects
Backend	Not configured (manual/cloud VM)	Manual setup
Database	Local or MongoDB Atlas	MONGO_URI in .env

6.4 Why React?

React provides component reusability (Card, Layout, Profile), a large ecosystem, hooks for state and side effects, and industry adoption. React 19 with Vite offers fast development with hot module replacement during implementation.

6.5 Why MongoDB?

Farm records vary in shape (animals vs milk vs expenses). Document storage avoids rigid relational migrations during prototyping. Mongoose adds schema validation while preserving flexibility for fields like optional profile images stored as long strings.

6.6 Why Express?

Express is minimal, widely taught, and integrates cleanly with JWT middleware and Mongoose. Express 5 is used in this project for modern routing and middleware patterns aligned with current Node.js LTS releases.

Chapter 7 — Implementation: Backend

7.1 Server Bootstrap (server.js)

The entry file performs the following steps in sequence:

- `dotenv.config()` — loads environment variables.
- Creates Express app instance.
- Applies `cors()` globally.
- Parses JSON bodies up to 10 MB (supports base64 profile images).
- Connects MongoDB via `mongoose.connect(process.env.MONGO_URI)`.
- Mounts six route prefixes.
- Defines health route `GET /`.
- Listens on `process.env.PORT`.

7.2 Authentication Routes

POST `/api/auth/register`

Accepts name, email, and password. Password is bcrypt-hashed (cost factor 10) before persisting. Returns `{ message: "User registered" }`.

POST `/api/auth/login`

Verifies credentials; issues JWT signed with `JWT_SECRET`, expiring in 1 day. Returns `{ token, name }`.

7.3 Route Summary

Route			
<code>/api/users/profile</code>	GET	Yes	Returns name, email, image
<code>/api/users/upload</code>	PUT	Yes	Updates image field (base64)
<code>/api/animals</code>	POST	Yes	Create animal
<code>/api/animals</code>	GET	Yes	List user animals
<code>/api/animals/:id</code>	DELETE	Yes	Remove animal
<code>/api/milk</code>	POST	Yes	Log milk record
<code>/api/milk</code>	GET	Yes	List milk records
<code>/api/milk/:id</code>	DELETE	Yes	Remove milk record
<code>/api/vaccinations</code>	POST	Yes	Record vaccination
<code>/api/vaccinations</code>	GET	Yes	List vaccinations

Route			
/api/vaccinations/:id	DELETE	Yes	Remove vaccination
/api/expenses	POST	Yes	Record expense/profit
/api/expenses	GET	Yes	List expenses
/api/expenses/:id	DELETE	Yes	Remove expense
/	GET	No	Health check — plain text

7.4 Authentication Middleware

The protect middleware extracts the Bearer token from the Authorization header, verifies it with `jwt.verify`, sets `req.user = decoded payload`, and returns 401 for missing or invalid tokens. All non-auth routes pass through this middleware.

7.5 Sample Register Request/Response

```
POST /api/auth/register
Content-Type: application/json

{ "name": "Ramesh", "email": "ramesh@gmail.com", "password": "****" }

Response: { "message": "User registered" }
```

7.6 Animal Create — Field Mapping

JSON body key		
id	animalId	Farmer-defined tag
breed	breed	From dropdown
age	age	String in schema
gender	gender	Male/Female/Other
cost	cost	Purchase price
health	health	e.g. Healthy, Sick
(auto)	user	Set from JWT req.user.id

7.7 Error Handling Pattern

Route handlers use try/catch blocks. Errors return HTTP 500 with `{ message: error.message }`. Login failures return 400 with user-friendly messages. Missing JWT returns 401 from protect middleware. There is no centralized error middleware; improvement for production would use Express error handler.

Chapter 7 (continued) — Backend Details

7.8 Middleware Execution Order

For every protected request, Express applies `cors()` and `express.json()` globally first. The route-specific `protect` middleware runs before the `async` handler. If verification succeeds, `req.user` contains the decoded JWT payload (at minimum the user id). Handlers never trust client-supplied user ids in the body — ownership is always taken from the token.

7.9 Collection Names in MongoDB

Mongoose pluralizes model names by default: `User` becomes `users`, `Animal` becomes `animals`, `Milk` becomes `milks`, `Vaccination` becomes `vaccinations`, `Expense` becomes `expenses`. Compass or Atlas UI will show these collection names when inspecting the database after first save.

7.10 Recommended Production Middleware

- `express-rate-limit` on `/api/auth/login` to reduce brute force.
- `helmet()` for secure HTTP headers.
- `express-validator` or `Zod` for request body schema validation.
- `morgan("combined")` for access logging.
- Centralized error handler returning consistent JSON error shape.

7.11 Dead Code Warning

The file `backend/controllers/authcontroller.js` contains alternate `register/login` using hardcoded JWT secret `"SECRET123"` and a `role` field not present in the `User` schema. This file is not mounted in `server.js`. Remove or merge it before production to avoid confusion during security review or viva questions.

Chapter 8 — Implementation: Frontend

8.1 Application Routing (App.jsx)

Path		
<code>/</code>	Landing	No
<code>/login</code>	Login	No
<code>/register</code>	Register	No
<code>/forgot-password</code>	ForgotPassword	No

Path		
/farmer-dashboard	FarmerDashboard	Yes
/department-dashboard	DepartmentDashboard	Yes
/animals	Animal	Yes
/milk	Milk	Yes
/vaccination	Vaccination	Yes
/expenses	Expenses	Yes

8.2 Layout Shell

- **Topbar** — menu hover, language toggle, dark mode toggle, profile dropdown.
- **Sidebar** — NavLink; hidden until hover (open state).
- **Footer** — rendered on authenticated pages.
- **Outlet** — child route content.

8.3 API Integration Pattern

Every protected page constructs requests using the farmer's JWT token from localStorage. A recommended improvement is a centralized apiClient.js using VITE_API_URL for the base path to avoid hardcoded localhost:5000 across all pages.

8.4 Unused / Partial Modules

File	
services/api.js	Axios stub pointing to api.example.com — not used
services/storage.js	localStorage CRUD — superseded by API
context/LanguageContext	Partially redundant with App-level state management

8.5 Login Page Behavior

Login.jsx posts to http://localhost:5000/api/auth/login. On success it saves token, user name, and auth=true, then navigates to farmer-dashboard. Password visibility toggle uses local show state. Errors display in red below the email field.

8.6 Register Page Validation

- Phone number must be exactly 10 digits (UI validation only; not sent to API).
- Email must end with @gmail.com (client-side rule).
- Name and password read from form placeholders.

8.7 Internationalization

translations.js exports text.en and text.hi objects with 100+ keys for sidebar, dashboard, forms, and landing page. App.jsx holds lang state and persists to localStorage. Topbar toggles EN/Hi. LanguageContext exists but is not correctly wired in main.jsx — language primarily flows via props.

8.8 Chart.js Dashboard Integration

FarmerDashboard registers CategoryScale, LinearScale, BarElement, Tooltip, Legend. Weekly milk data aggregates records by weekday into a 7-element array for the bar chart. Chart label uses translated milkProduction string.

Chapter 9 — Database Design

9.1 Entity Relationships

A User document has a one-to-many relationship with Animals, Milk records, Vaccinations, and Expenses. Each child document stores a user field (MongoDB ObjectId) referencing User._id to enforce per-user data isolation.

9.2 Collections Overview

User Collection

Field		
_id	ObjectId	Primary key
name	String	Display name
email	String	Login identifier
password	String	bcrypt hash
image	String	Optional base64/data URL

Animal Collection

Field			
user	ObjectId	Yes	Owner reference
animalId	String	Yes	Farmer-defined tag e.g. A1
breed	String	Yes	Dropdown values
age	String	No	Stored as string from form
gender	String	No	Male/Female/Other
cost	String	No	Purchase cost
health	String	No	e.g. Healthy, Sick
createdAt	Date	Auto	Timestamps

Vaccination Collection

Field		
user	ObjectId	Owner
animalId	String	Target animal
breed	String	Breed at time of vaccination

Field		
vaccineName	String	FMD, HS, BQ, etc.
date	String	Administration date
upcomingDate	String	Next due or 'No Booster' text

9.3 Indexing Recommendations (Future)

- Unique compound index on (user, animalId) to prevent duplicate animal IDs per farmer.
- Descending index on (user, date) for milk records to speed date-range queries.

9.4 Milk Collection

Field		
user	ObjectId	Owner reference
animalId	String	Links to animal tag
date	String	YYYY-MM-DD from date input
quantity	Number	Litres

9.5 Expense Collection

Field		
user	ObjectId	Owner
type	String	Expense or Profit
category	String	Feed, Milk Sale, etc.
amount	Number	INR
date	String	Transaction date
description	String	Optional notes

9.6 Sample Seed Data (data/ folder)

Animals.json contains 10 sample entries (Jersey, Gir, Sahiwal, etc.) with id A1–A10. These files supported early UI mockups before MongoDB integration. They are not imported by server.js.

Chapter 10 — Security & Authentication

10.1 Threat Model

Threat		
Password theft from DB	bcrypt hashing	Weak user passwords still possible
XSS stealing JWT	None specific	localStorage vulnerable
IDOR (cross-user data)	user filter on queries	Good mitigation
Brute force login	None	Add rate limiting
Secret leakage	.env for JWT	Must not commit secrets

10.2 Password Hashing

Algorithm: **bcrypt** with cost factor **10**. The hash is stored in the User document; compare is done on login via `bcrypt.compare(password, user.password)`.

10.3 JWT Lifecycle

- 1. Issued at login with payload { id: user._id }.
- 2. Client stores in localStorage key "token".
- 3. Sent as Authorization: Bearer on protected routes.
- 4. Server verifies with `process.env.JWT_SECRET`.
- 5. Expires after 1 day — client does not auto-refresh.

10.4 Security Checklist for Production

- [] HTTPS everywhere
- [] Restrict CORS to frontend origin
- [] HttpOnly cookie option instead of localStorage
- [] Input validation (Joi/Zod)
- [] Rate limit /api/auth/login
- [] Rotate JWT secrets
- [] Add .env to .gitignore on backend
- [] Helmet.js for HTTP headers

10.5 Authorization vs Authentication

Authentication (protect middleware) verifies the JWT and identifies the user. **Authorization** is enforced by querying only documents where user field equals `req.user.id`. There is no admin role or

cross-tenant read in the current codebase.

10.6 Profile Image Considerations

- Client limits upload to 2 MB before base64 encoding.
- Entire image stored in User.image string — simple but increases document size.
- Production should use S3, Cloudinary, or GridFS with URL reference only.

Chapter 11 — User Interface & Screen Documentation

11.1 Landing Page (/)

- Hero — Title, tagline, Get Started, Watch Demo
- Stats — Marketing numbers (farms, milk tracked, farmers)
- Features Grid — 6 cards: Animal Registration, Vaccine Tracking, Milk Production, etc.
- Benefits — Save time, increase milk, control expenses, mobile access
- How It Works — 3 steps
- Testimonial — Star rating and quote
- FAQ — Two question/answer pairs
- Footer — Links and copyright text

11.2 Farmer Dashboard (/farmer-dashboard)

KPI Cards

Card	
Total Animals	animals.length
Today Milk	Sum quantities where date === today
Revenue Today	todayMilk * 70
Total Milk	Sum all quantities
Top Animal	Max milk by animalId
Farm Score	(totalMilk * 0.5) + (totalAnimals * 2) + (revenueToday * 0.1)

11.3 Animal Management (/animals)

Field	
Animal ID	Text input
Breed	Select: Gir, Sahiwal, Red Sindhi, Tharparkar, HF Cross, Jersey, Murrah Buffalo, etc.
Age	Number input
Gender	Select: Male, Female, Other
Cost	Number input (displayed with Rs. symbol)
Health	Text input

11.4 Vaccination Schedule (/vaccination)

Vaccine	
FMD	6 months
HS	12 months
BQ	12 months
Brucellosis	No booster
Theileriosis	No booster
Anthrax	12 months

11.5 Expenses & Profit (/expenses)

Expense categories: Feed, Veterinary, Labor, Equipment, Maintenance, Breeding, Animal Purchase. Profit categories: Milk Sale, Dairy Products, Animal Sale, Manure Sale, Government Subsidy. Summary row shows totals and net profit/loss in green or red.

11.6 Login Page (/login)

Branded as CattleCloud with aquamarine accent styling. Fields: email (required), password with show/hide toggle. Links to forgot-password and register. No role selection; all users route to farmer-dashboard after login.

11.7 Register Page (/register)

Collects farmer name, email, phone, password. Client validates Gmail suffix and 10-digit phone. Submits only name, email, password to backend.

11.8 Forgot Password (/forgot-password)

Important limitation: This flow reads users from localStorage, not MongoDB. It generates a 6-digit code client-side and updates local user array. It does not work for real registered accounts unless users were duplicated into localStorage manually. Production requires email OTP API.

11.9 Milk Production Page (/milk)

Element	
Form	Animal ID, date picker, quantity in litres
Table	Per-row income = quantity × Rs. 70
Footer row	Total litres and total income
Below table	Average milk per record

11.10 Expense Categories Detail

Expense types

- Feed
- Veterinary
- Labor
- Equipment
- Maintenance
- Breeding
- Animal Purchase

Profit types

- Milk Sale
- Dairy Products
- Animal Sale
- Manure Sale
- Government Subsidy

11.11 Department Dashboard

Route /department-dashboard shows static cards: 120 farms, 3200 animals, 12000L milk. No sidebar link; not connected to backend aggregation APIs.

11.12 Profile & Topbar

Profile.jsx fetches /api/users/profile, shows dropdown with upload (max 2MB), logout. Topbar provides menu hover for sidebar, language toggle, dark mode on body class, Profile component.

11.13 Navigation Map

Route	
/farmer-dashboard	Dashboard KPIs and chart
/animals	Herd registry
/milk	Production log
/vaccination	Vaccines and reminders
/expenses	Farm accounting

11.14 Landing Page Sections (Detailed)

- **Hero:** Title, bilingual tagline, Get Started → register, Watch Demo YouTube link, Explore Features scroll.
- **Stats:** Marketing metrics: farms registered, milk tracked, farmers served.
- **Trusted By:** Dairy, agriculture, milk producers, community labels.
- **Features:** Six cards: animal registration, vaccine tracking, milk production, breeding, vet support, expenses.
- **Dashboard preview:** Screenshot-style image from Unsplash.
- **Benefits:** Save time, increase milk, control expenses, access anywhere.
- **How it works:** Three step cards: register, add animals, track daily.
- **Testimonial:** Five-star quote and CTA buttons.
- **FAQ:** Two Q&A; items about cost and data safety.
- **Final CTA:** Start free registration button.

11.15 UI Styling Approach

Global CSS classes in index.css and App.css define landing page hero, feature grids, tables (.styled-table, .form-grid), and layout (.main-content, .topbar-fixed). Many authenticated pages mix className with inline style objects for rapid iteration. Dark mode toggles a class on document.body from Topbar. Color palette uses greens for agriculture branding and light gray table row backgrounds for readability.

11.16 Known UI Gaps

Gap	
Dashboard vaccines=[]	Vaccination KPI cards always zero
No edit buttons	User must delete and re-add to fix mistakes
Sidebar hover-only	Poor mobile discoverability
Hardcoded API URL	Production build cannot reach backend
Forgot password local	Misleading for real users

Chapter 12 — Algorithms & Business Logic

12.1 Today's Milk Calculation

```
const today = new Date().toISOString().split('T')[0];
const todayMilk = milkRecords
  .filter(m => m.date === today)
  .reduce((sum, m) => sum + Number(m.quantity), 0);
```

12.2 Revenue Estimation

```
const milkPrice = 70; // INR per litre (hardcoded)
const revenueToday = todayMilk * milkPrice;
```

12.3 Farm Performance Score

```
const performanceScore =
  (totalMilk * 0.5) +
  (totalAnimals * 2) +
  (revenueToday * 0.1);
```

Heuristic scoring for gamification-style display — not an industry standard metric.

12.4 Weekly Milk Aggregation for Chart

```
const days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"];
const weeklyData = new Array(7).fill(0);
milkRecords.forEach(r => {
  const d = new Date(r.date).getDay(); // 0=Sun
  const index = d === 0 ? 6 : d - 1; // Mon=0
  weeklyData[index] += Number(r.quantity);
});
```

12.5 Vaccination Booster Date

```
if (months > 0) {
  const nextDate = new Date(selectedDate);
  nextDate.setMonth(nextDate.getMonth() + months);
  upcomingDate = nextDate.toISOString().split('T')[0];
} else {
  upcomingDate = "No Booster";
}
```

12.6 Vaccine Due Reminder

```
const diff = (new Date(v.upcomingDate) - new Date())  
/ (1000 * 60 * 60 * 24);  
return diff <= 3; // flag if due within 3 days
```

12.7 Net Farm Profit/Loss

```
const totalExpenses = records.filter(r => r.type === "Expense")  
.reduce((sum, r) => sum + Number(r.amount), 0);  
const totalProfit = records.filter(r => r.type === "Profit")  
.reduce((sum, r) => sum + Number(r.amount), 0);  
const netProfitLoss = totalProfit - totalExpenses;
```

12.8 Top Animal Selection

Milk records are grouped by animalId in a hash map; entries sorted descending by total quantity; first entry displayed as top producer on dashboard.

12.9 Average Milk on Milk Page

$\text{avgMilk} = \text{totalMilk} / (\text{record count} || 1)$. Displayed below table with two decimal places.

Chapter 13 — API Reference

Base URL (development): `http://localhost:5000`

Auth header (protected routes): `Authorization: Bearer <token>`

Method			
POST	<code>/api/auth/register</code>	No	Register user; body: { name, email, password }
POST	<code>/api/auth/login</code>	No	Login; returns { token, name }
GET	<code>/api/users/profile</code>	Yes	Returns { name, email, image }
PUT	<code>/api/users/upload</code>	Yes	Update profile image (base64)
POST	<code>/api/animals</code>	Yes	Create animal
GET	<code>/api/animals</code>	Yes	List user's animals
GET	<code>/api/animals/:id</code>	Yes	Single animal
DELETE	<code>/api/animals/:id</code>	Yes	Delete animal
POST	<code>/api/milk</code>	Yes	Log milk record
GET	<code>/api/milk</code>	Yes	List milk records
DELETE	<code>/api/milk/:id</code>	Yes	Delete milk record
POST	<code>/api/vaccinations</code>	Yes	Record vaccination
GET	<code>/api/vaccinations</code>	Yes	List vaccinations
DELETE	<code>/api/vaccinations/:id</code>	Yes	Delete vaccination
POST	<code>/api/expenses</code>	Yes	Record expense/profit
GET	<code>/api/expenses</code>	Yes	List expenses
DELETE	<code>/api/expenses/:id</code>	Yes	Delete expense
GET	<code>/</code>	No	Health check — 'Backend running'

HTTP Status Codes Used

Code	
200	Success
201	Animal created
400	Login credential errors
401	Missing/invalid JWT

Code	
500	Server/database errors

13.2 Sample Login Response

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "name": "Ramesh Kumar"
}
```

13.3 Sample Milk POST Body

```
{ "animalId": "A101", "date": "2026-05-19", "quantity": 12 }
```

Chapter 14 — Testing & Validation

14.1 Testing Strategy

Formal automated tests are not implemented (npm test exits with error on backend). Validation was performed through manual test cases during development.

14.2 Test Cases

TC			
TC-01	User Registration	Open /register; enter valid Gmail + password; submit	New document in users collection with hashed password
TC-02	Login	POST correct credentials / wrong password	JWT returned / Error message shown
TC-03	Animal CRUD	Add, refresh, delete, switch user	Persists; other user cannot see data
TC-04	Milk Records	Add 10L for today; check dashboard	Income = Rs.700; dashboard KPI updates
TC-05	Vaccination Booster	Add FMD; check next due date	Date ~6 months after administration
TC-06	Expenses	Add expense Rs.1000; add profit Rs.5000	Net = Rs.4000 displayed
TC-07	Profile Image	Upload image <2MB; reload	Preview and API return same image
TC-08	Language Toggle	Switch EN to HI; reload	Sidebar in Hindi; preference persisted

14.3 Negative Testing

Case	
API call without token	401 Unauthorized
Malformed JSON body	500 or parser error
Duplicate registration email	500 / error message

14.4 Test Environment

Component	
OS	Windows 10/11
Browser	Chrome / Edge latest
Backend	http://localhost:5000
Frontend	http://localhost:5173 (Vite)
Database	Local MongoDB or Atlas

14.5 Integration Test Flow (End-to-End)

- Register → Login → Add animal A1 → Add milk 15L today → Dashboard shows updated today milk.
- Add FMD vaccination → Verify upcomingDate six months ahead.
- Add expense Rs.2000 feed → Add profit Rs.8000 milk sale → Net profit Rs.6000.
- Logout → Attempt /animals → Redirect to login.

Chapter 15 — Deployment & DevOps

15.1 Backend Deployment Steps

- 1. Provision VM or PaaS (Railway, Render, AWS EC2).
- 2. Set environment variables: MONGO_URI, JWT_SECRET, PORT.
- 3. Run `npm install --production` and `npm start`.
- 4. Configure firewall to expose port.
- 5. Optionally use PM2 for process management.

15.2 Frontend Deployment (Netlify)

```
[build]
command = "npm run build"
publish = "dist"

[[redirects]]
from = "/*"
to = "/index.html"
status = 200
```

15.3 Environment Variables

Backend .env

```
MONGO_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/cattlecloud
JWT_SECRET=<long-random-string>
PORT=5000
```

Frontend .env

```
VITE_API_URL=https://your-backend.example.com
```

15.4 Production Gap

Netlify-hosted frontend still calls `localhost:5000` unless `VITE_API_URL` is introduced. Backend must be deployed separately with HTTPS and CORS restricted to the Netlify domain.

15.5 Backup Strategy

Enable MongoDB Atlas automated backups. Export collections before schema changes. Document restore procedure for viva demonstration recovery.

Chapter 16 — Results & Discussion

16.1 Achieved Outcomes

- Persistent cloud storage of farm records.
- Visual dashboard for milk analytics with Chart.js bar chart.
- Vaccination scheduling with due-soon warnings.
- Bilingual interface (English/Hindi) suitable for Indian farmers.
- Secure password storage and JWT-protected APIs with per-user data boundaries.

16.2 Demonstration Script (Viva)

Step		
1	Show landing page and language toggle	1 min
2	Register new farmer account	1 min
3	Login and show dashboard	1 min
4	Add 2 animals with different breeds	2 min
5	Add milk records; refresh dashboard — KPI change	2 min
6	Add vaccination; show next due date and alert banner	1 min
7	Add expense and profit; show net summary	1 min
8	Profile image upload	1 min
9	Mention department dashboard as future work	30 sec

16.3 Learning Outcomes

- REST API design and HTTP verbs.
- MongoDB schema design and references.
- React hooks and conditional rendering.
- Authentication flows in SPAs.
- CORS and environment configuration.
- Static site deployment.

16.4 Discussion

The project demonstrates digitization themes in agriculture while remaining achievable in one semester. Trade-offs favor development speed over enterprise patterns (DTO validation, repository layer, microservices). Chart-based feedback may encourage daily milk logging—a behavioral benefit beyond

storage alone.

16.5 Learning Outcomes (Extended)

- Design RESTful resources aligned with domain entities.
- Implement bcrypt and JWT in Node.js.
- Build protected routes in React Router v7.
- Use Mongoose schemas with ObjectId references.
- Configure SPA deployment with fallback redirects.
- Document limitations honestly for academic integrity.

Chapter 17 — Limitations & Known Issues

17.1 Functional Limitations

#		
L1	Forgot password not connected to MongoDB	High
L2	Dashboard ignores vaccination API	Medium
L3	No edit (update) endpoints	Medium
L4	Department dashboard is static	Medium
L5	Phone number collected but not stored	Low
L6	Gmail-only registration rule	Low

17.2 Technical Bugs / Risks

#		
B1	Orphan LanguageProvider JSX	main.jsx
B2	Layout import wrong case	App.jsx
B3	Hardcoded API URL	All pages
B4	token not cleared on logout	Topbar.jsx, Layout.jsx
B5	Unused authcontroller.js with weak secret	backend/controllers/
B6	PrivateRoute ignores token validity	PrivateRoute.jsx

17.3 Operational Limitations

- Requires internet for API; no offline PWA cache.
- No automated backup UI for farmer-exported data.
- Milk price hardcoded — not regional or market-linked.
- No SMS/push for vaccine due dates.

17.4 Ethical Considerations

Farmer data stored in cloud database — privacy policy and consent flow not included in app. Recommend terms of use before any public production launch.

Chapter 18 — Future Scope

18.1 Short-Term Enhancements (1–2 months)

- Fix main.jsx and Layout import casing.
- Introduce VITE_API_URL across all fetch calls.
- Connect dashboard to /api/vaccinations.
- Implement server-side password reset (email OTP).
- Clear all auth keys on logout; redirect on 401 globally.
- Add unique compound index on (user, animalId).

18.2 Medium-Term Features (3–6 months)

- Role-based access — farmer vs department with aggregated read APIs.
- Edit records — PUT/PATCH for animals, milk, etc.
- PDF/Excel export of milk and expense reports.
- SMS reminders for vaccines (Twilio / MSG91).
- PWA — service worker for limited offline entry.
- Mobile responsive audit — improve sidebar UX on phones.

18.3 Long-Term Vision (6+ months)

- IoT integration — automatic milk meter readings.
- GIS mapping — farm location and veterinary routing.
- Marketplace module — connect farmers to buyers.
- AI yield prediction from historical milk data.
- Integration with government livestock ID schemes.
- Microservices split if user base scales significantly.

18.4 Integration Roadmap

Phase 1: stability fixes (main.jsx, API URL, logout token clear). Phase 2: department APIs and password reset. Phase 3: IoT and AI analytics. This phased approach matches typical startup MVP evolution.

Chapter 19 — Conclusion

CattleCloud demonstrates a complete **concept-to-implementation** pipeline for a livestock management web application. By combining a React frontend, Express backend, and MongoDB persistence, the project meets core academic objectives: requirements analysis, system design, modular implementation, and user-centered features for dairy farmers.

The system's strongest engineering aspects are **consistent user-scoped APIs**, a **coherent domain model** (animals → milk → health → finance), and **bilingual usability**. Areas requiring further work — production configuration, password recovery, department analytics, and automated testing — provide a clear roadmap beyond the minor project submission.

With the enhancements outlined in Chapter 18, CattleCloud could evolve from an academic MVP into a deployable rural SaaS product contributing to smarter, data-driven livestock farming.

19.1 Summary of Deliverables

- Working farmer portal with five data modules.
- REST API with six route groups and JWT protection.
- MongoDB persistence with five collections.
- Bilingual marketing and application UI.
- Netlify-ready frontend build configuration.
- This project report and generated PDF documentation.

Appendix A — Project Directory Structure

```
minorproject/
  ■■■ PROJECT_REPORT.md
  ■■■ backend/
    ■ ■■■ server.js
    ■ ■■■ package.json
    ■ ■■■ .env (local secrets – do not publish)
    ■ ■■■ controllers/
    ■ ■ ■■■ authcontroller.js (unused)
    ■ ■■■ middleware/
    ■ ■ ■■■ authMiddleware.js
    ■ ■■■ models/
    ■ ■ ■■■ User.js
    ■ ■ ■■■ Animal.js
    ■ ■ ■■■ Milk.js
    ■ ■ ■■■ Vaccination.js
    ■ ■ ■■■ Expense.js
    ■ ■■■ routes/
    ■ ■■■ authRoutes.js
    ■ ■■■ animalRoutes.js
    ■ ■■■ milkRoutes.js
    ■ ■■■ vaccinationRoutes.js
    ■ ■■■ expenseRoutes.js
  ■■■ smart-livestock-dashboard/
    ■ ■■■ index.html
    ■ ■■■ vite.config.js
    ■ ■■■ netlify.toml
    ■ ■■■ src/
    ■ ■■■ main.jsx
    ■ ■■■ App.jsx
    ■ ■■■ components/
    ■ ■■■ context/
    ■ ■■■ pages/
    ■ ■■■ services/
    ■ ■■■ utils/translations.js
  ■■■ data/
  ■■■ Animals.json
  ■■■ Milk.json
  ■■■ vaccination.json
  ■■■ expenses.json
```

Appendix B — Glossary

Term	
API	Application Programming Interface; HTTP endpoints for data exchange
bcrypt	Password hashing function resistant to brute force
CORS	Cross-Origin Resource Sharing; browser security for APIs
CRUD	Create, Read, Update, Delete operations
FMD	Foot and Mouth Disease — common cattle vaccination
JWT	JSON Web Token — signed credential for stateless auth
KPI	Key Performance Indicator — dashboard metric
Mongoose	MongoDB object modeling library for Node.js
ODM	Object Document Mapper
REST	Representational State Transfer — HTTP-based API style
SPA	Single Page Application — client-side routed web app
Vite	Frontend build tool and development server

Appendix C — References

- [1] React Documentation — <https://react.dev>
- [2] Express.js Guide — <https://expressjs.com>
- [3] Mongoose Documentation — <https://mongoosejs.com>
- [4] MongoDB Manual — <https://www.mongodb.com/docs>
- [5] JSON Web Tokens Introduction — <https://jwt.io/introduction>
- [6] Chart.js Documentation — <https://www.chartjs.org/docs>
- [7] Vite Guide — <https://vite.dev>
- [8] Netlify SPA Redirects — <https://docs.netlify.com/routing/redirects/>
- [9] OWASP Authentication Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- [10] MDN Fetch API — https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Appendix D — Viva Preparation Question Bank

Basic

Q1. What is CattleCloud?

A: A web-based livestock management system for recording animals, milk, vaccinations, and expenses.

Q2. Which database did you use and why?

A: MongoDB — flexible schema for varied farm records, easy integration with Node via Mongoose.

Q3. How does login work?

A: Server verifies bcrypt hash, returns JWT; client stores token and sends Bearer header on API calls.

Intermediate

Q4. How do you prevent Farmer A from seeing Farmer B's animals?

A: Every query includes { user: req.user.id } from decoded JWT.

Q5. Explain the vaccination booster logic.

A: A rules map defines months per vaccine; frontend adds months to administration date for upcomingDate.

Q6. What is the farm score formula?

A: $(\text{totalMilk} * 0.5) + (\text{totalAnimals} * 2) + (\text{revenueToday} * 0.1)$ — heuristic display metric.

Advanced

Q7. What are JWT drawbacks in localStorage?

A: Vulnerable to XSS; httpOnly cookies are a safer alternative.

Q8. How would you deploy frontend and backend together?

A: Host API on HTTPS subdomain; set VITE_API_URL; build React to static hosting; configure CORS.

Q9. Why is forgot password not production-ready?

A: It updates localStorage users, not MongoDB; no email verification.

Q10. How would you add an edit animal feature?

A: Implement PUT /api/animals/:id with findOneAndUpdate filtered by user, add Edit button in UI.

Appendix F — Team Contribution Template

Member		
Student A		Frontend: Landing, Login, Dashboard
Student B		Frontend: Animal, Milk pages
Student C		Backend: Auth, Animal APIs
Student D		Backend: Milk, Vaccination, Expenses APIs
Student E		Documentation, Testing, Deployment

(Fill before submission)

Appendix E — Translation Module

File: smart-livestock-dashboard/src/utils/translations.js exports const text = { en: {...}, hi: {...} }.

Category	
Sidebar	dashboard, animals, milk, vaccination, expenses
Dashboard	totalAnimals, todayMilk, weeklyMilk, farmScore
Animal	animalManagement, searchAnimal, addAnimal
Milk	milkProductionTitle, milkQuantity, averageMilk
Landing	heroTitle1, keyFeatures, faq, ctaTitle

Usage pattern

```
const t = text[lang] || text['en'];
return <h2>{t.animalManagement}</h2>;
```

Appendix G — How to Run the Project

- 1. Install Node.js LTS and MongoDB.
- 2. cd backend → create .env with MONGO_URI, JWT_SECRET, PORT=5000.
- 3. npm install && npm start → verify GET / returns Backend running.
- 4. cd smart-livestock-dashboard → npm install && npm run dev.
- 5. Open browser URL from Vite (usually :5173) → register and login.

Appendix H — Extended Viva Questions

Q: What is MERN?

A: MongoDB, Express, React, Node — our stack uses Vite instead of CRA for React.

Q: Difference between SQL and MongoDB here?

A: We use flexible documents; no JOINS; user field links ownership.

Q: What is CORS?

A: Browser security; server uses cors() to allow frontend origin in dev.

Q: How is password stored?

A: bcrypt hash only; never plain text in database.

Q: What happens when JWT expires?

A: API returns 401; frontend should redirect to login (improvement area).

Q: Explain Chart.js usage.

A: Bar chart of weekly milk totals on farmer dashboard.

Q: What is PrivateRoute?

A: React Router wrapper checking localStorage auth flag.

Q: Why Netlify redirects?

A: SPA needs index.html fallback for client routes like /animals.